

1. Deep-Dive Mathematical Derivation from First Principles

In Nomura's Cash Equities Central Risk Book (CRB) team, an Executive Director will use this question to test your ability to distinguish between raw statistical metrics and operational, transaction-cost-adjusted portfolio returns. In statistical arbitrage and quantitative execution, a daily out-of-sample R^2 of 5% is extraordinarily large. If a researcher claims to have achieved this on liquid equities, it almost always signals look-ahead bias, data leakage, or failure to account for microstructural decay.

Below is the step-by-step mathematical derivation linking predictive power (R^2) to the maximum attainable Information Ratio (IR), followed by the modeling of signal decay and forecast calibration under physical turnover constraints.

Phase A: Line-by-Line Link Between R^2 and Information Ratio (IR)

Let r_t be the realized excess return of a cash equity asset at day t , and let $\hat{r}_t = \mathbb{E}[r_t \mid \mathcal{F}_{t-1}]$ be the conditional linear forecast generated by our alpha model given the information set $\mathcal{F}_{t-1}$. We decompose the true return as:

$$r_t = \hat{r}_t + \epsilon_t$$

Where ϵ_t is an idiosyncratic white-noise residual satisfying $\mathbb{E}[\epsilon_t] = 0$, $\text{Var}(\epsilon_t) = \sigma_\epsilon^2$, and $\text{Cov}(\hat{r}_t, \epsilon_t) = 0$.

Step 1: Variance Decomposition from First Principles

Because $\hat{r}_t$ and ϵ_t are orthogonal by construction, the total variance of the asset return $\sigma_r^2 = \text{Var}(r_t)$ expands linearly:

$$\begin{aligned}\text{Var}(r_t) &= \text{Var}(\hat{r}_t + \epsilon_t) = \text{Var}(\hat{r}_t) + \text{Var}(\epsilon_t) + 2\text{Cov}(\hat{r}_t, \epsilon_t) \\ \sigma_r^2 &= \sigma_{\hat{r}}^2 + \sigma_\epsilon^2\end{aligned}$$

Step 2: Defining R^2 as Variance Explained

The coefficient of determination R^2 is the ratio of the variance of our conditional forecast to the total variance of the asset's returns:

$$R^2 = \frac{\sigma_{\hat{r}}^2}{\sigma_r^2} = \frac{\sigma_{\hat{r}}^2}{\sigma_{\hat{r}}^2 + \sigma_\epsilon^2}$$

We solve for the signal variance $\sigma_{\hat{r}}^2$ in terms of R^2 and σ_ϵ^2 :

$$\begin{aligned}R^2(\sigma_{\hat{r}}^2 + \sigma_\epsilon^2) &= \sigma_{\hat{r}}^2 \\ R^2\sigma_{\hat{r}}^2 + R^2\sigma_\epsilon^2 &= \sigma_{\hat{r}}^2 \\ R^2\sigma_\epsilon^2 &= \sigma_{\hat{r}}^2(1 - R^2) \\ \sigma_{\hat{r}}^2 &= \frac{R^2}{1 - R^2}\sigma_\epsilon^2\end{aligned}$$

Taking the square root of both sides gives the standard deviation of our prediction:

$$\sigma_{\hat{r}} = \sqrt{\frac{R^2}{1 - R^2}} \sigma_{\epsilon}$$

Step 3: Deriving Daily Information Ratio via Optimal Sizing

Suppose we trade a single strategy where our active position allocation w_t tracks the alpha forecast directly: $w_t = \hat{r}_t$. The daily dollar PnL of this signal is $\pi_t = w_t r_t = \hat{r}_t(\hat{r}_t + \epsilon_t) = \hat{r}_t^2 + \hat{r}_t \epsilon_t$.

The daily expected PnL of this unconstrained allocation is:

$$\mathbb{E}[\pi_t] = \mathbb{E}[\hat{r}_t^2] + \mathbb{E}[\hat{r}_t \epsilon_t] = \sigma_{\hat{r}}^2 + 0 = \sigma_{\hat{r}}^2$$

To calculate the daily Information Ratio (IR_{daily}), we find the daily variance of this PnL. Since asset returns are dominated by noise, the variance of the PnL is driven primarily by the residual variance σ_{ϵ}^2 acting on the position size:

$$\text{Var}(\pi_t) \approx \mathbb{E}[w_t^2] \sigma_{\epsilon}^2 = \mathbb{E}[\hat{r}_t^2] \sigma_{\epsilon}^2 = \sigma_{\hat{r}}^2 \sigma_{\epsilon}^2$$

The daily Information Ratio is defined as the expected PnL divided by its standard deviation:

$$IR_{\text{daily}} = \frac{\mathbb{E}[\pi_t]}{\sqrt{\text{Var}(\pi_t)}} = \frac{\sigma_{\hat{r}}^2}{\sigma_{\hat{r}} \sigma_{\epsilon}} = \frac{\sigma_{\hat{r}}}{\sigma_{\epsilon}}$$

Substitute the expression for $\sigma_{\hat{r}}$ from Step 2 into this ratio:

$$IR_{\text{daily}} = \frac{\sqrt{\frac{R^2}{1 - R^2}} \sigma_{\epsilon}}{\sigma_{\epsilon}} = \sqrt{\frac{R^2}{1 - R^2}}$$

Step 4: Annualizing via the Breadth Coefficient (BR)

Assuming N independent, identical asset trading opportunities per day across the book's cross-section, and 252 trading days per year, the annualized Breadth (BR) equals $N \times 252$. Aggregating these independent returns scales the Information Ratio by $\sqrt{BR}$:

$$IR_{\text{annualized}} = IR_{\text{daily}} \times \sqrt{BR} = \sqrt{\frac{R^2}{1 - R^2}} \times \sqrt{N \times 252}$$

For a single asset ($N = 1$) with the proposed daily $R^2 = 5\% = 0.05$:

$$IR_{\text{annualized}} = \sqrt{\frac{0.05}{1 - 0.05}} \times \sqrt{252} = \sqrt{\frac{0.05}{0.95}} \times \sqrt{252} \approx 0.2294 \times 15.8745 \approx 3.64$$

An annualized Information Ratio of 3.64 before transaction costs on a single liquid equity name is extraordinarily high. This explains why an institutional risk manager will view an in-sample daily $R^2 = 5\%$ with extreme skepticism.

Phase B: Mathematical Modeling of Signal Decay over Multi-Day Horizons

Signals in cash equities decay as the market incorporates the underlying information into prices. We model this decay using a first-order autoregressive continuous-time proxy. Let α_t represent the latent alpha signal state, which decays exponentially over a discrete rebalancing interval Δt :

$$\alpha_{t+\Delta t} = \alpha_t e^{-\lambda \Delta t} + \eta_{t+\Delta t}$$

Where $\lambda > 0$ is the structural decay parameter (half-life $\tau = \frac{\ln 2}{\lambda}$), and η is an independent innovation vector.

Step 1: Formulating the Multi-Period Information Loss Function

If the central risk book rebalances at an interval of Δt , the average effective alpha captured by the book over that horizon is found by integrating the decay curve:

$$\bar{\alpha}(\Delta t) = \frac{1}{\Delta t} \int_0^{\Delta t} \alpha_t e^{-\lambda s} ds = \alpha_t \left[\frac{1 - e^{-\lambda \Delta t}}{\lambda \Delta t} \right]$$

The coefficient $\Phi(\Delta t) = \frac{1 - e^{-\lambda \Delta t}}{\lambda \Delta t}$ acts as an information discount factor. As rebalancing frequency decreases (Δt grows), $\Phi(\Delta t) \rightarrow 0$.

Step 2: Incorporating Transaction Costs Into the Rebalancing Objective

Frequent trading reduces information loss but increases transaction costs. We model execution costs as a quadratic function of turnover: $\mathcal{C}(\Delta \mathbf{w}) = \frac{\gamma}{2} (\Delta \mathbf{w})^\top \mathbf{\Lambda} (\Delta \mathbf{w})$, where $\mathbf{\Lambda}$ represents the market impact matrix and $\Delta \mathbf{w} = \mathbf{w}_t - \mathbf{w}_{t-\Delta t}$.

The optimization problem over a horizon Δt balances the decayed expected return against execution costs:

$$\max_{\mathbf{w}_t} \mathcal{J}(\mathbf{w}_t) = \mathbf{w}_t^\top \left(\alpha_t \left[\frac{1 - e^{-\lambda \Delta t}}{\lambda \Delta t} \right] \right) - \frac{\gamma}{2} (\mathbf{w}_t - \mathbf{w}_{t-\Delta t})^\top \mathbf{\Lambda} (\mathbf{w}_t - \mathbf{w}_{t-\Delta t})$$

Step 3: First-Order Optimality Conditions

To solve for the optimal portfolio weights, we take the gradient of our objective function $\mathcal{J}(\mathbf{w}_t)$ with respect to $\mathbf{w}_t$ and set it to zero:

$$\begin{aligned} \nabla_{\mathbf{w}_t} \mathcal{J} &= \alpha_t \left[\frac{1 - e^{-\lambda \Delta t}}{\lambda \Delta t} \right] - \gamma \mathbf{\Lambda} (\mathbf{w}_t - \mathbf{w}_{t-\Delta t}) = \mathbf{0} \\ \gamma \mathbf{\Lambda} (\mathbf{w}_t - \mathbf{w}_{t-\Delta t}) &= \alpha_t \left[\frac{1 - e^{-\lambda \Delta t}}{\lambda \Delta t} \right] \end{aligned}$$

Step 4: Solving for the Optimal Rebalancing Step Vector

Assuming a simplified diagonal market impact matrix ($\mathbf{\Lambda} = \lambda_0 \mathbf{I}$), we isolate the trade execution vector:

$$\mathbf{w}_t^* = \mathbf{w}_{t-\Delta t} + \frac{1}{\gamma \lambda_0} \left[\frac{1 - e^{-\lambda \Delta t}}{\lambda \Delta t} \right] \alpha_t$$

This equation demonstrates the optimal trade-off: if a signal decays rapidly (large λ), the information discount factor shrinks. This forces the central risk book to scale down its execution sizes to avoid consuming excess liquidity through high-turnover trades.

Phase C: Implementation of Forecast Calibration (The Reality Check)

To ensure that raw statistical alpha forecasts do not imply unphysical turnover or require liquidity beyond market capacity, we apply a mathematical calibration layer known as the **Campbell-Thompson Sign Constraint** and a **Turnover Penalty Filter**.

Step 1: The Campbell-Thompson Constraint Mapping

The Campbell & Thompson (2008) operational constraint states that an alpha forecast can only be used if its sign matches the expected directional sign, and it must be set to zero if it contradicts the basic risk premium assumption:

$$\hat{r}_t^{\text{calibrated}} = \hat{r}_t \cdot \mathbb{I}_{\{\hat{r}_t \cdot \text{sign}(\text{Risk Premium}) > 0\}}$$

Step 2: The Turnover-Constrained Forecast Transformation

We apply an optimization-free penalty filter directly to the raw forecast vector to bound implied turnover. Let $\tau_{\max}$ be the maximum allowable daily turnover percentage for the book. The calibrated forecast vector $\tilde{\mathbf{r}}_t$ is constrained by:

$$\tilde{\mathbf{r}}_t = \arg \min_{\mathbf{x}} \|\mathbf{x} - \hat{\mathbf{r}}_t\|_2^2 \quad \text{subject to} \quad \sum_{i=1}^d |x_i - w_{i,t-1}| \leq \tau_{\max}$$

Using Lagrange multipliers, the regularized, unconstrained objective function can be written as:

$$\tilde{\mathbf{r}}_t = \mathbf{w}_{t-1} + \text{SoftThreshold}(\hat{\mathbf{r}}_t - \mathbf{w}_{t-1}, \rho)$$

Where ρ is a tuning parameter mapped directly to the liquidity capacity of the asset's cross-section. This mathematical filter ensures that raw statistical predictions are calibrated to match the physical execution constraints of the central risk book before entering the portfolio construction pipeline.

2. Quantitative Infrastructure & Application of Theory

The following production-grade Python script implements this complete analytical pipeline. It simulates a cash equity cross-section, runs an exponential signal decay engine, computes transaction costs from first principles, applies the Campbell-Thompson validation checks, and builds an interactive dashboard using Plotly to show how R^2 , decay speed, and turnover constraints interact.

```
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# =====
# 1. GENERATE DECAYING ALPHA SIGNALS & ASSET UNIVERSE
# =====
def generate_crb_signal_decay_data(
    n_assets: int = 30, n_days: int = 252,
    true_r2: float = 0.005, half_life_days: float = 2.0, seed: int = 42
) -> tuple:
    """
```

```

Simulates an institutional asset universe with realistic out-of-sample R^2
and exponential alpha signal decay profiles.
"""
np.random.seed(seed)
timeline = pd.date_range(start="2025-01-01", periods=n_days, freq='B')
tickers = [f"EQ_{i:03d}" for i in range(n_assets)]

# Map the structural decay parameter lambda from the specified half-life
lam = np.log(2.0) / half_life_days

# Generate historical daily asset return matrix (simulating underlying noise)
raw_returns = np.random.normal(loc=0.0002, scale=0.015, size=(n_days,
n_assets))

# Derive the required signal standard deviation to match the target R^2
# R^2 = var_signal / var_total -> var_signal = R^2 * var_total / (1 - R^2)
var_total = 0.015 ** 2
var_signal = (true_r2 * var_total) / (1.0 - true_r2)
std_signal = np.sqrt(var_signal)

# Construct the autoregressive AR(1) decaying alpha signal matrix
alpha_forecasts = np.zeros((n_days, n_assets))
alpha_forecasts[0, :] = np.random.normal(0, std_signal, size=n_assets)

for t in range(1, n_days):
    # Apply exponential decay step + new innovation
    innovation = np.random.normal(0, std_signal * np.sqrt(1.0 - np.exp(-2 *
lam))), size=n_assets)
    alpha_forecasts[t, :] = alpha_forecasts[t-1, :] * np.exp(-lam) +
innovation

    return timeline, tickers, raw_returns, alpha_forecasts, lam

# =====
# 2. REALITY CHECK FORECAST CALIBRATION ENGINE
# =====
class CRBForecastValidator:
    """
    Applies the Campbell-Thompson sign constraint and regularized turnover
    filters to calibrate raw statistical forecasts to physical execution limits.
    """
    def __init__(self, tickers: list, lam_decay: float, max_turnover: float =
0.15):
        self.tickers = tickers
        self.lam = lam_decay
        self.max_turnover = max_turnover
        self.n_assets = len(tickers)

    def apply_campbell_thompson_constraint(self, raw_forecast: np.ndarray) ->
np.ndarray:
    """
    Enforces that alpha forecasts must align with the historical sign
    of the asset's risk premium.
    """

```

```

        # Assume a positive baseline risk premium for long-term equity markets
        calibrated_forecast = np.where(raw_forecast > 0, raw_forecast, 0.0)
        return calibrated_forecast

    def apply_turnover_penalty_filter(self, current_forecast: np.ndarray,
current_weights: np.ndarray) -> np.ndarray:
        """
        Applies a regularized soft-thresholding filter to limit implied portfolio
turnover.
        """
        raw_delta = current_forecast - current_weights

        # Determine the dynamic shrinkage penalty threshold based on turnover
constraints
        shrinkage_penalty = 1.5e-4

        # Compute the analytical soft-thresholding operator
        delta_sign = np.sign(raw_delta)
        delta_magnitude = np.abs(raw_delta) - shrinkage_penalty
        delta_magnitude = np.where(delta_magnitude > 0, delta_magnitude, 0.0)

        calibrated_forecast = current_weights + (delta_sign * delta_magnitude)
        return calibrated_forecast

# =====
# 3. PRODUCTION BACKTEST & BACKBONE PORTFOLIO SIMULATOR
# =====
def run_crb_execution_backtest(
    timeline, tickers, returns, forecasts, lam, r2, max_turn,
transaction_cost_bps: float = 5.0
):
    """
    Runs the portfolio construction backtest loop, evaluating performance
with and without the forecast calibration layers.
    """
    n_days, n_assets = returns.shape
    validator = CRBForecastValidator(tickers, lam_decay=lam,
max_turnover=max_turn)

    # Initialize trackers for tracking weights
    weights_naive = np.zeros(n_assets)
    weights_calibrated = np.zeros(n_assets)

    pnl_naive_history = []
    pnl_calibrated_history = []
    turnover_naive_history = []
    turnover_calibrated_history = []

    cost_multiplier = transaction_cost_bps / 10000.0

    for t in range(n_days):
        raw_f = forecasts[t, :]
        realized_r = returns[t, :]

```

```

# --- NAIVE PIPELINE ---
# Allocate weights directly proportional to the raw signal
target_w_naive = raw_f / (np.sum(np.abs(raw_f)) + 1e-8)
turnover_naive = np.sum(np.abs(target_w_naive - weights_naive))

gross_pnl_naive = np.sum(weights_naive * realized_r)
tc_naive = turnover_naive * cost_multiplier
net_pnl_naive = gross_pnl_naive - tc_naive

# --- CALIBRATED PIPELINE ---
f_ct = validator.apply_campbell_thompson_constraint(raw_f)
f_reg = validator.apply_turnover_penalty_filter(f_ct, weights_calibrated)

target_w_calib = f_reg / (np.sum(np.abs(f_reg)) + 1e-8)
turnover_calib = np.sum(np.abs(target_w_calib - weights_calibrated))

gross_pnl_calib = np.sum(weights_calibrated * realized_r)
tc_calib = turnover_calib * cost_multiplier
net_pnl_calib = gross_pnl_calib - tc_calib

# Append performance stats
pnl_naive_history.append(net_pnl_naive)
pnl_calibrated_history.append(net_pnl_calib)
turnover_naive_history.append(turnover_naive)
turnover_calibrated_history.append(turnover_calib)

# Update portfolio states for the next time step
weights_naive = target_w_naive
weights_calibrated = target_w_calib

return (
    np.array(pnl_naive_history), np.array(pnl_calibrated_history),
    np.array(turnover_naive_history), np.array(turnover_calibrated_history)
)

# =====
# 4. INTERACTIVE PLOTLY DASHBOARD VISUALIZATION
# =====
def render_crb_decay_dashboard(timeline, pnl_n, pnl_c, turn_n, turn_c, r2, lam):
    """
    Renders an interactive multi-panel dashboard comparing cumulative net PnL,
    implied portfolio turnover, and drawdown profiles.
    """
    cum_pnl_n = np.cumsum(pnl_n)
    cum_pnl_c = np.cumsum(pnl_c)

    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=(
            "Cumulative Strategy Net PnL (After Transaction Costs)",
            "Daily Implied Portfolio Turnover Profile",
            "Information Ratio Sensitivity Horizon Mapping",
            "Strategy Risk Drawdown Analysis"
        ),
    ),

```

```

        vertical_spacing=0.15, horizontal_spacing=0.12
    )

    # Panel 1: Net Cumulative PnL Curves
    fig.add_trace(go.Scatter(x=timeline, y=cum_pnl_n, mode='lines',
        line=dict(color='crimson', width=2), name='Raw Naive Model'), row=1, col=1)
    fig.add_trace(go.Scatter(x=timeline, y=cum_pnl_c, mode='lines',
        line=dict(color='emerald', width=2.5), name='Calibrated Reality-Check Model'),
        row=1, col=1)

    # Panel 2: Portfolio Turnover Timeline
    fig.add_trace(go.Scatter(x=timeline, y=turn_n, mode='lines',
        line=dict(color='indianred', width=1), opacity=0.5, name='Raw Turnover'), row=1,
        col=2)
    fig.add_trace(go.Scatter(x=timeline, y=turn_c, mode='lines',
        line=dict(color='mediumseagreen', width=1.5), name='Calibrated Turnover'), row=1,
        col=2)

    # Panel 3: Theoretical IR vs. Empirical R^2 Frontier Map
    r2_space = np.linspace(0.001, 0.05, 100)
    ir_analytical_1day = np.sqrt(r2_space / (1.0 - r2_space)) * np.sqrt(252)
    ir_analytical_5day = np.sqrt(r2_space / (1.0 - r2_space)) * np.sqrt(252 / 5.0)
    # Accounting for decay

    fig.add_trace(go.Scatter(x=r2_space * 100, y=ir_analytical_1day, mode='lines',
        line=dict(color='cyan', width=2), name='Theoretical IR (Daily Rebal)'), row=2,
        col=1)
    fig.add_trace(go.Scatter(x=r2_space * 100, y=ir_analytical_5day, mode='lines',
        line=dict(color='gold', width=2, dash='dash'), name='Theoretical IR (Weekly
        Rebal)'), row=2, col=1)
    fig.add_trace(go.Scatter(x=[r2 * 100], y=[np.sqrt(r2 / (1.0 - r2)) *
        np.sqrt(252)], mode='markers', marker=dict(size=12, color='orange',
        symbol='star'), name='Active SDE Target Point'), row=2, col=1)

    # Panel 4: Strategy Drawdown Comparison
    def compute_drawdown(cum_pnl):
        cum_max = np.maximum.accumulate(cum_pnl)
        # Avoid dividing by zero if maximum cumulative PnL is zero
        cum_max = np.where(cum_max == 0, 1.0, cum_max)
        return cum_pnl - cum_max

    fig.add_trace(go.Scatter(x=timeline, y=compute_drawdown(cum_pnl_n),
        mode='lines', line=dict(color='purple'), name='Naive Drawdown'), row=2, col=2)
    fig.add_trace(go.Scatter(x=timeline, y=compute_drawdown(cum_pnl_c),
        mode='lines', line=dict(color='dodgerblue'), name='Calibrated Drawdown'), row=2,
        col=2)

    # Layout and Axis Configuration
    fig.update_layout(
        title_text=f"Nomura Cash Equities CRB – Signal Decay Framework Dashboard
        (Target Daily R² = {r2*100}%)",
        title_x=0.5, height=900, width=1300, template="plotly_dark",
        showlegend=True
    )

```



```

fig.update_xaxes(title_text="Trading Calendar Timeline", row=1, col=1)
fig.update_yaxes(title_text="Cumulative Net Returns (PnL Score)", row=1,
col=1)
fig.update_xaxes(title_text="Trading Calendar Timeline", row=1, col=2)
fig.update_yaxes(title_text="Daily Portfolio Swaps (Turnover Scale)", row=1,
col=2)
fig.update_xaxes(title_text="Out-of-Sample Predictive R2 Value (%)", row=2,
col=1)
fig.update_yaxes(title_text="Annualized Information Ratio (IR)", row=2, col=1)
fig.update_xaxes(title_text="Trading Calendar Timeline", row=2, col=2)
fig.update_yaxes(title_text="Peak-to-Trough Capital Eradication Scale", row=2,
col=2)

fig.show()

# =====
# 5. ENTRY POINT EXECUTION ENGINE GATEWAY
# =====
if __name__ == "__main__":
    # Configure the test parameters to match the interview problem
    target_r2_metric = 0.005      # Standard institutional target: 0.5% out-of-
sample R^2
    signal_half_life_days = 1.5   # Rapidly decaying alpha signal: 1.5 day half-
life
    portfolio_assets_count = 50
    trading_days_horizon = 252
    max_allowable_turnover = 0.10 # Limit daily turnover to 10%
    execution_slippage_bps = 8.0  # Execution slippage cost: 8 bps

    # 1. Simulate the assets and signal decay matrices
    dates, assets, asset_returns, signal_forecasts, decay_constant =
generate_crb_signal_decay_data(
    n_assets=portfolio_assets_count, n_days=trading_days_horizon,
    true_r2=target_r2_metric, half_life_days=signal_half_life_days, seed=101
)

    # 2. Run the historical simulation backtest
    pnl_raw, pnl_calib, turn_raw, turn_calib = run_crb_execution_backtest(
        dates, assets, asset_returns, signal_forecasts,
        decay_constant, target_r2_metric, max_turnover=max_allowable_turnover,
        transaction_cost_bps=execution_slippage_bps
    )

    # 3. Calculate metrics for the summary report
    ir_raw_empirical = (np.mean(pnl_raw) / (np.std(pnl_raw) + 1e-8)) *
np.sqrt(252)
    ir_calib_empirical = (np.mean(pnl_calib) / (np.std(pnl_calib) + 1e-8)) *
np.sqrt(252)

    print("\n" + "="*65)
    print("CRB SIGNAL REALITY CHECK PIPELINE REPORT:")
    print("="*65)
    print(f"Configured Out-of-Sample Predictive R2 : {target_r2_metric*100:.2f}%")

```

```

print(f"Modeled Alpha Decay Constant (Lambda) : {decay_constant:.4f}")
print(f"Assigned Execution Slippage Penalty : {execution_slippage_bps:.1f}
bps")
print("-"*65)
print(f"Raw Model Empirical Annualized IR : {ir_raw_empirical:.4f}")
print(f"Calibrated Model Annualized IR : {ir_calib_empirical:.4f}")
print("-"*65)
print(f"Raw Model Mean Daily Turnover :
{np.mean(turn_raw)*100:.2f}%")
print(f"Calibrated Model Mean Daily Turnover :
{np.mean(turn_calib)*100:.2f}%")
print("="*65 + "\n")

# 4. Render the diagnostic visualization platform
render_crb_decay_dashboard(dates, pnl_raw, pnl_calib, turn_raw, turn_calib,
target_r2_metric, decay_constant)

```

3. Strategic Interview Follow-Ups & Edge-Case Vulnerabilities

Follow-Up 1: Distinguishing Between Real Alpha and Data Sponging

Question: "You have derived the link showing that a daily $R^2 = 0.5\%$ can yield an annualized Information Ratio above 1.0. However, if your predictive R^2 is computed over the same data sample used to extract your model parameters, why does this statistical performance metric fail to guarantee trading profitability? How do you mathematically protect the central risk book from data mining or data sponging?"

- **The Answer:** An in-sample R^2 is an overly optimistic measure of true predictive power because standard optimization techniques minimize residuals specifically on the training data. This leads to a severe upward bias known as **overfitting** or data mining.
- **The Mathematical Vulnerability:** If you run an Ordinary Least Squares (OLS) regression with K parameters on an equity time-series containing T historical days, the expected in-sample R^2 , even when the underlying data is completely random white noise with zero predictive relationship, is given by:

$$\mathbb{E}[R^2_{\text{in-sample}}] = \frac{K - 1}{T - 1}$$

- If a researcher tests 100 features ($K = 101$) on one year of daily data ($T = 252$), the model will show an expected in-sample R^2 of $\frac{100}{251} \approx 39.8\%$ entirely due to random chance, while its true out-of-sample R^2 is zero or negative.
- **The Solution for the Book:** To protect the book's capital, we must enforce a multi-layered validation framework:
 1. **Out-of-Sample Adjusted R^2 Validation:** We evaluate model performance using out-of-sample datasets or cross-validation techniques that preserve the temporal ordering of time-series data (e.g., forward-chaining rolling windows).

2. **The Penalty Matrix:** We penalize model complexity by incorporating regularized objectives during parameter estimation, using Akaike Information Criterion (AIC) or Bayesian Information Criterion (BIC) constraints:

$$BIC = K \ln(T) - 2 \ln(\hat{L})$$

Where $\hat{L}$ represents the maximized value of the model's likelihood function. This ensures that a feature is only added if its marginal predictive power exceeds the statistical penalty for increasing the model's degrees of freedom.

Follow-Up 2: The Impact of Execution Slippage on Fast-Decaying Signals

Question: "Suppose your alpha model identifies a highly profitable signal with a very short half-life ($\tau = 10$ minutes). The model's raw statistical performance metrics are strong. However, when we attempt to execute the trade, we find that our realized Information Ratio drops below zero. Explain the microstructural components of this performance degradation. How do execution slippage and market impact vary as functions of your trading speed?"

- **The Answer:** For fast-decaying signals, execution performance is limited by a strict trade-off: trading too slowly causes the signal's information value to decay before the order is filled, while trading too quickly incurs high market impact costs that consume the strategy's expected returns.
- **The Mathematical Microstructure Decomposition:** We model the realized return of our trade net of microstructural costs as:

$$r_{\text{realized}}(t_{\text{exec}}) = \alpha_0 e^{-\lambda t_{\text{exec}}} - \mathcal{I}(V, t_{\text{exec}})$$

Where α_0 is the initial value of the signal, t_{exec} is the duration of our execution window, λ is the signal's decay rate, and $\mathcal{I}$ is the total transaction cost function.

- The total transaction cost combines fixed execution slippage (such as the half-spread cost) and a dynamic market impact component that depends on trading speed:

$$\mathcal{I}(V, t_{\text{exec}}) = \frac{\text{Bid-Ask Spread}}{2} + \theta \cdot \left(\frac{V}{t_{\text{exec}} \cdot \text{ADV}} \right)^\gamma$$

Where V is our total trade size, ADV is the stock's Average Daily Volume, θ is the structural market impact scale coefficient, and $\gamma \approx 0.5$ captures the empirical square-root scaling rule of execution costs.

- **The Operational Consequence:** This formula highlights the risk profile:
- If we trade instantly ($t_{\text{exec}} \rightarrow 0$) to capture the signal before it decays, the market impact term $\left(\frac{V}{t_{\text{exec}}} \right)^\gamma$ diverges toward infinity, and the high execution costs eliminate our alpha profits.
- If we trade slowly ($t_{\text{exec}} \rightarrow \infty$) to minimize market impact, the execution cost drops, but the signal value $\alpha_0 e^{-\lambda t_{\text{exec}}}$ decays to zero.
- To manage this effectively within a central risk book, we cannot treat alpha generation and trade execution as separate processes. The book must implement an **integrated execution scheduling engine** that optimizes the trading path t_{exec}^* by finding the exact point where the marginal cost of market impact matches the marginal loss from signal decay.